

Reti Neurali Ricorrenti

1. L'importanza del contesto

L'importanza del contesto

- Ricordati la 5° cifra del tuo numero di telefono
- Canta la tua canzone preferita iniziando alla terza frase
- Ricordati la 10° lettera dell'alfabeto

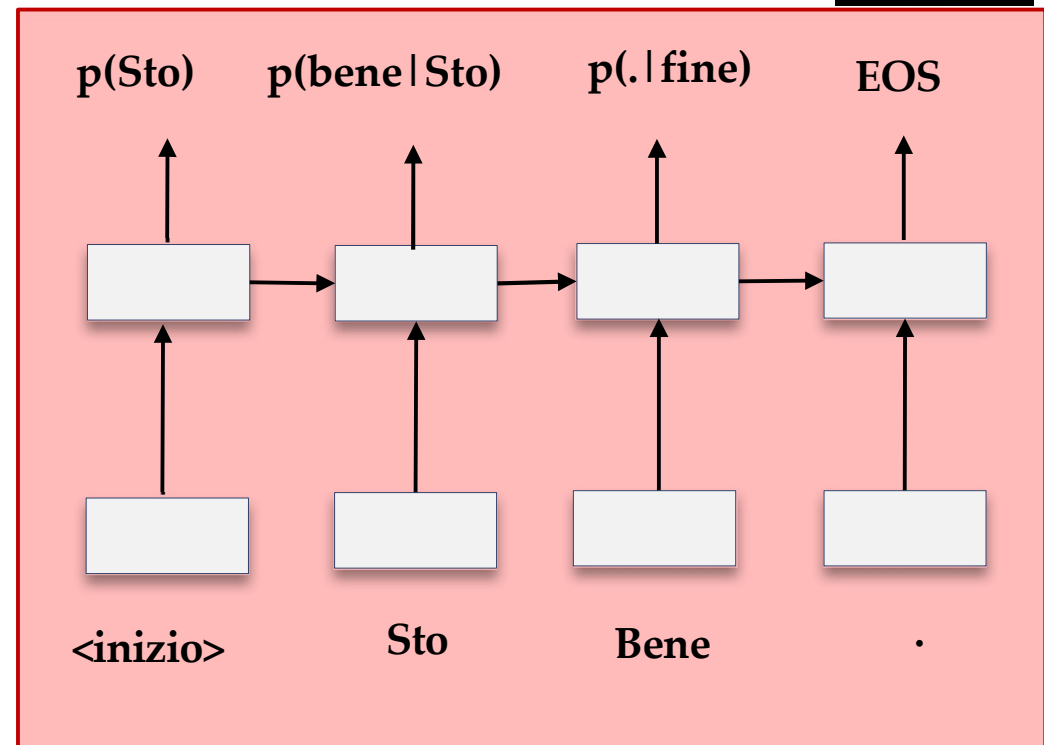
Probabilmente siete partiti dall'inizio ogni volta... perché nelle sequenze l'ordine è importante!

Idea: conservare l'informazione preservando l'ordine

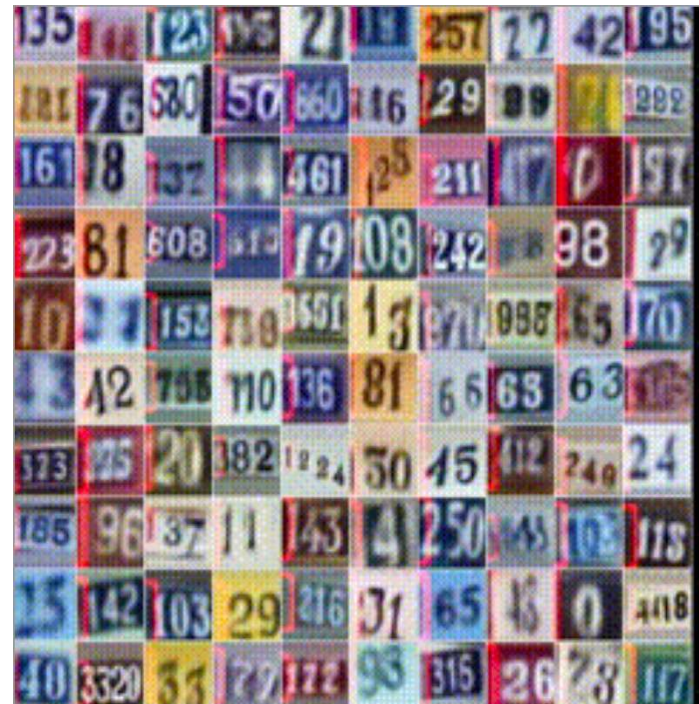
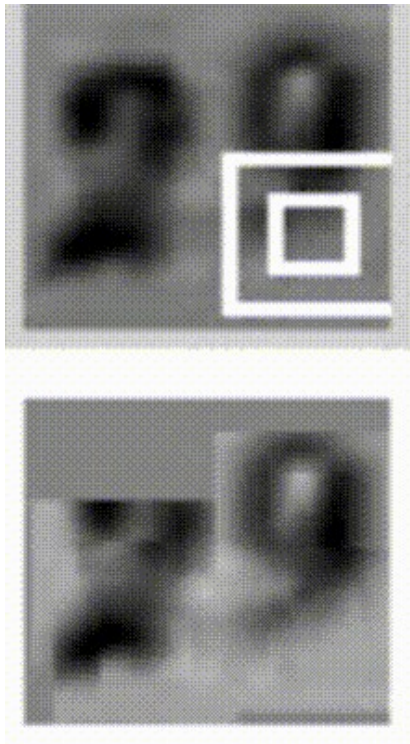
Applicazioni nel linguaggio



- Modellazione del linguaggio (probabilità)
- Traduzione automatica
- Riconoscimento del parlato



Applicazioni nelle immagini



2. Recap delle reti feedforward

Reti feedforward

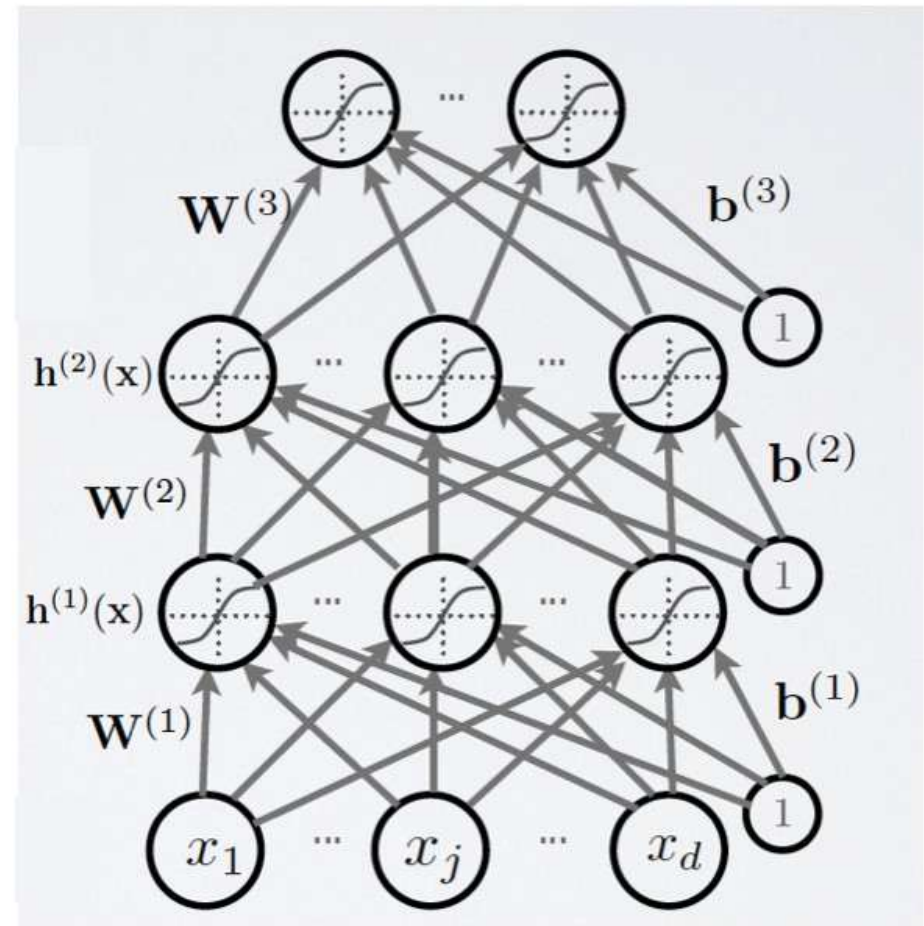
Ogni y/h_i È calcolato dalla sequenza delle attivazioni in avanti in output da $\mathbf{X}$.

$$y = f(\mathbf{W}_3 \cdot \mathbf{h}_2 + \mathbf{b}_3)$$

$$\mathbf{h}_2 = f(\mathbf{W}_2 \cdot \mathbf{h}_1 + \mathbf{b}_2)$$

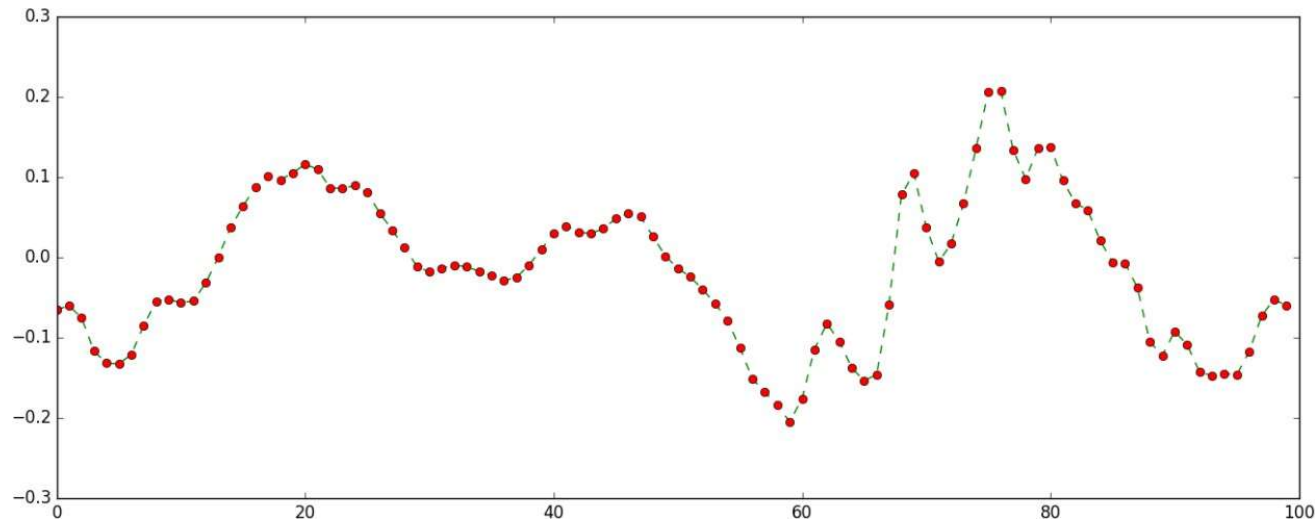
$$\mathbf{h}_1 = f(\mathbf{W}_1 \cdot \mathbf{x} + \mathbf{b}_1)$$

FEED FORWARD



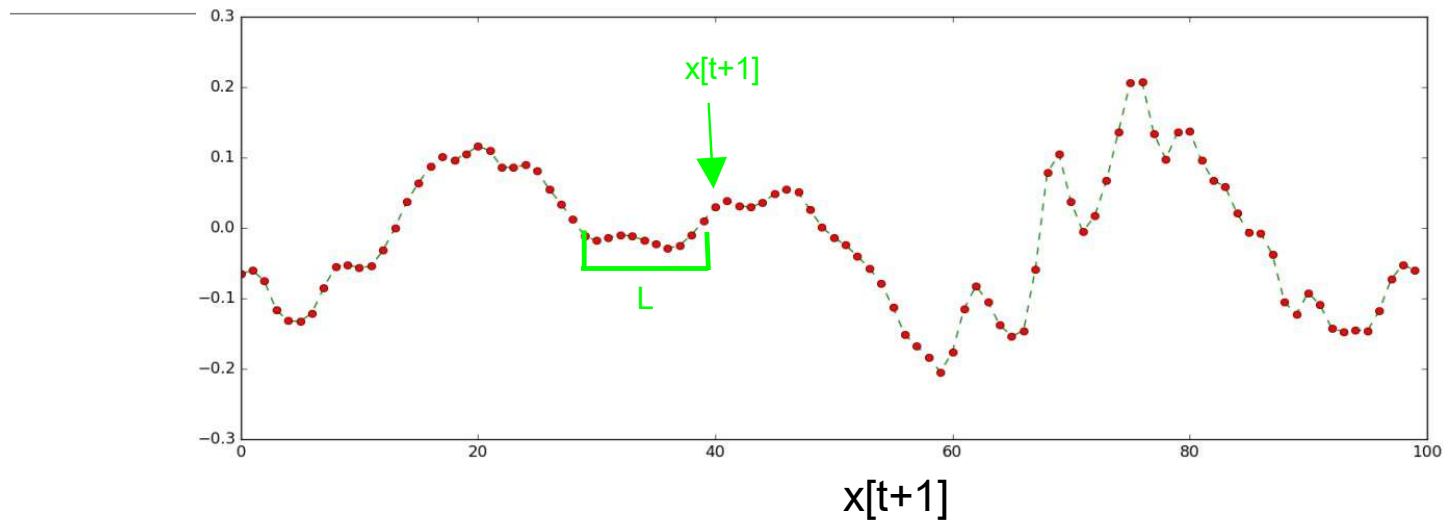
Dov'è la memoria?

Se abbiamo una sequenza di esempi...



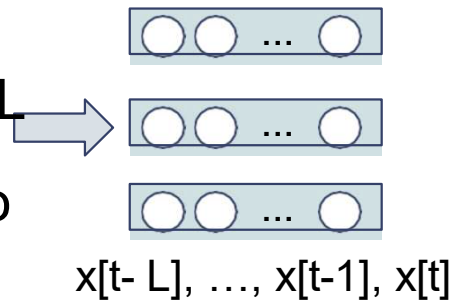
predire l'esempio $x[t+1]$ sapendo i valori precedenti $\{x[t], x[t-1], x[t-2], \dots, x[t-T]\}$

Dov'è la memoria?

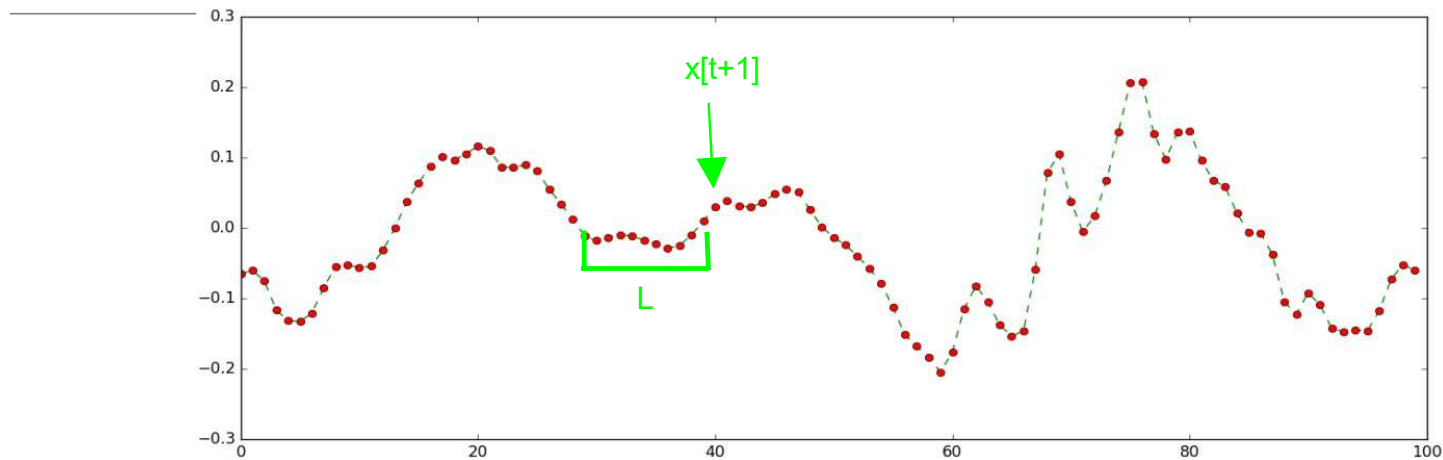


Approccio feed forward:

- finestra statica di dimensioni L
- Fai scorrere la finestra passo dopo passo

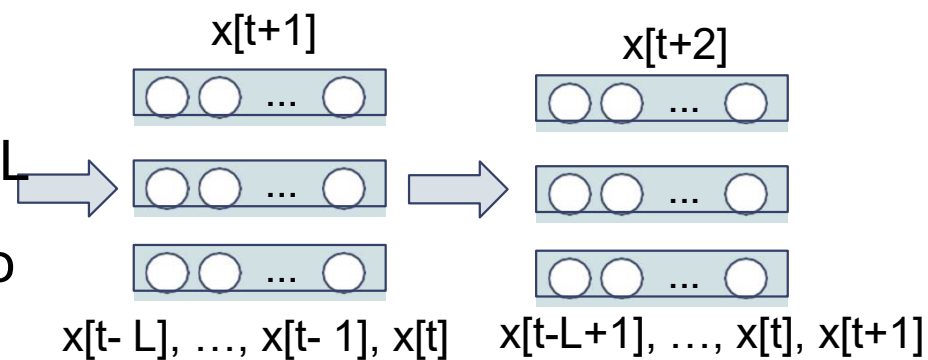


Dov'è la memoria?

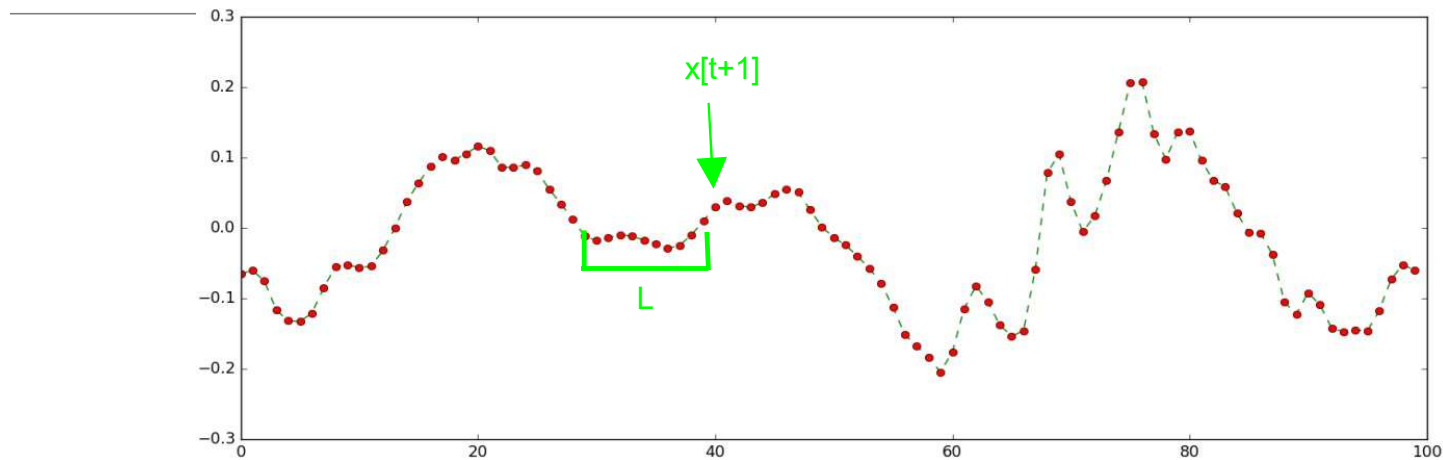


Approccio feed forward:

- finestra statica di dimensioni L
- Fai scorrere la finestra passo dopo passo

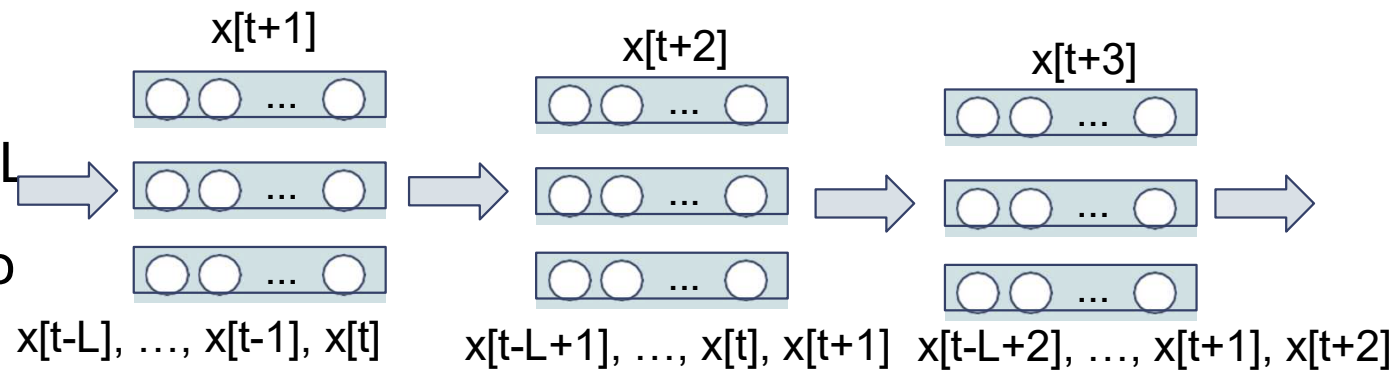


Dov'è la memoria?



Approccio feed forward:

- finestra statica di dimensioni L
- Fai scorrere la finestra passo dopo passo



Qualche problema con
questo approccio?

Problemi dell'approccio FFN + finestra statica

- Se L aumenta, rapida crescita del numero di parametri!

Problemi dell'approccio FFN + finestra statica

- Se L aumenta, veloce crescita del numero di parametri!
- Le decisioni sono indipendenti fra **timestep**!
 - Alla rete non importa cosa è successo a timestep precedenti, ma solo ciò che è accaduto nella presente finestra → non promette bene

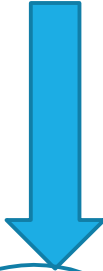
Problemi dell'approccio FFN + finestra statica

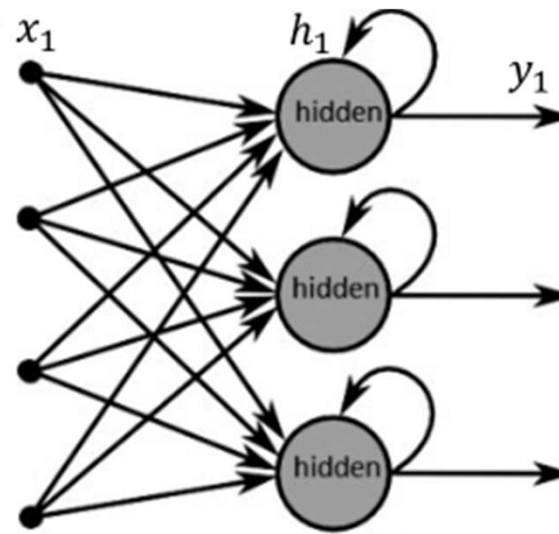
- Se L aumenta, veloce crescita del numero di parametri!
- Le decisioni sono indipendenti fra **timestep**!
 - Alla rete non importa cosa è successo a timestep precedenti, ma solo ciò che è accaduto nella presente finestra → non promette bene
- Padding ingombrante quando non ci sono abbastanza esempi da riempire L
 - Non può lavorare con sequenze di lunghezza variabile

3. RNN

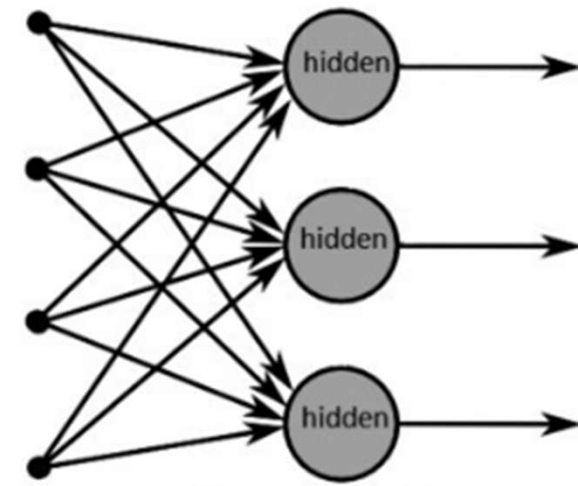
Rete Neurale Ricorrente (RNN) aggiungendo l'evoluzione "temporale"

Permette di costruire connessioni specifiche che catturano la "storia"


$$h_t = f(W \cdot x_t + U \cdot h_{t-1} + b)$$



(a) Recurrent neural network



(b) Forward neural network

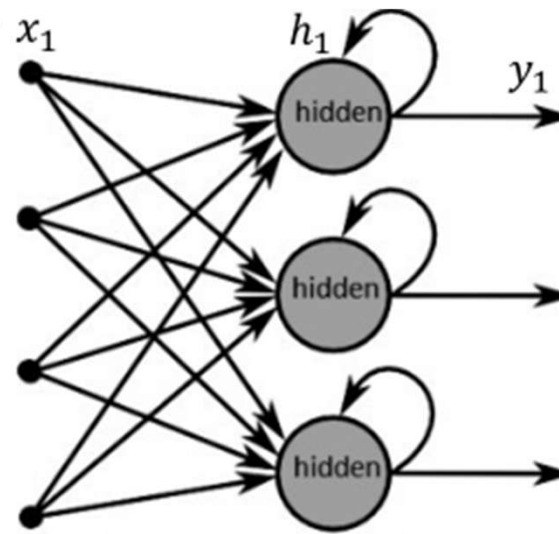
Rete Neurale Ricorrente (RNN) aggiungendo l'evoluzione "temporale"

Permette di costruire connessioni specifiche che catturano la "storia"

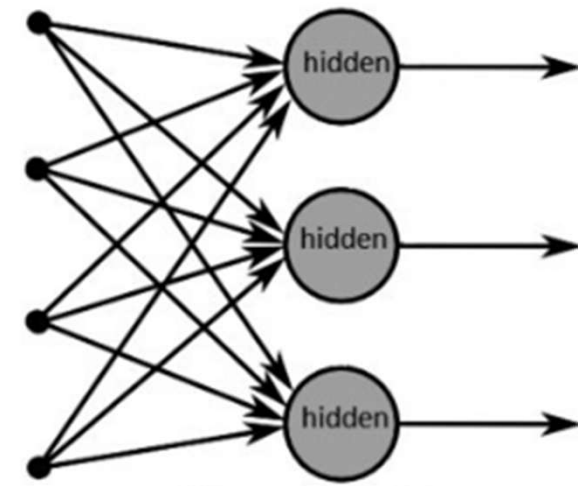


$$h_t = f(W \cdot x_t + U \cdot h_{t-1} + b)$$

$$y_t = \text{softmax}(Vh_t)$$



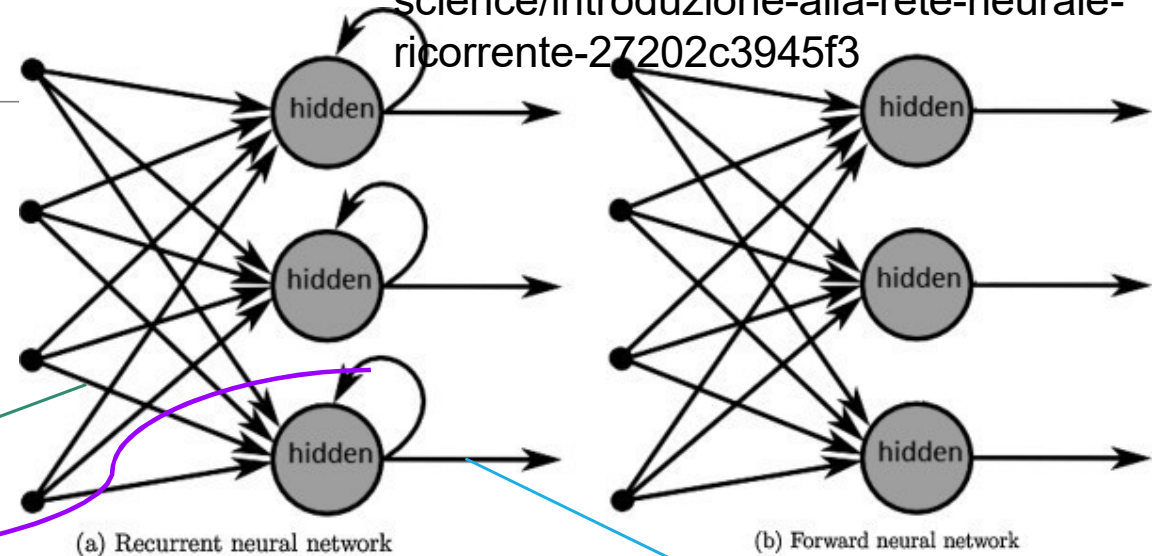
(a) Recurrent neural network



(b) Forward neural network

RNN: parametri

Immagine origine:
<https://medium.com/towards-data-science/introduzione-alla-rete-neurale-ricorrente-27202c3945f3>



$$h_t = f(W \cdot x_t + U \cdot h_{t-1} + b)$$

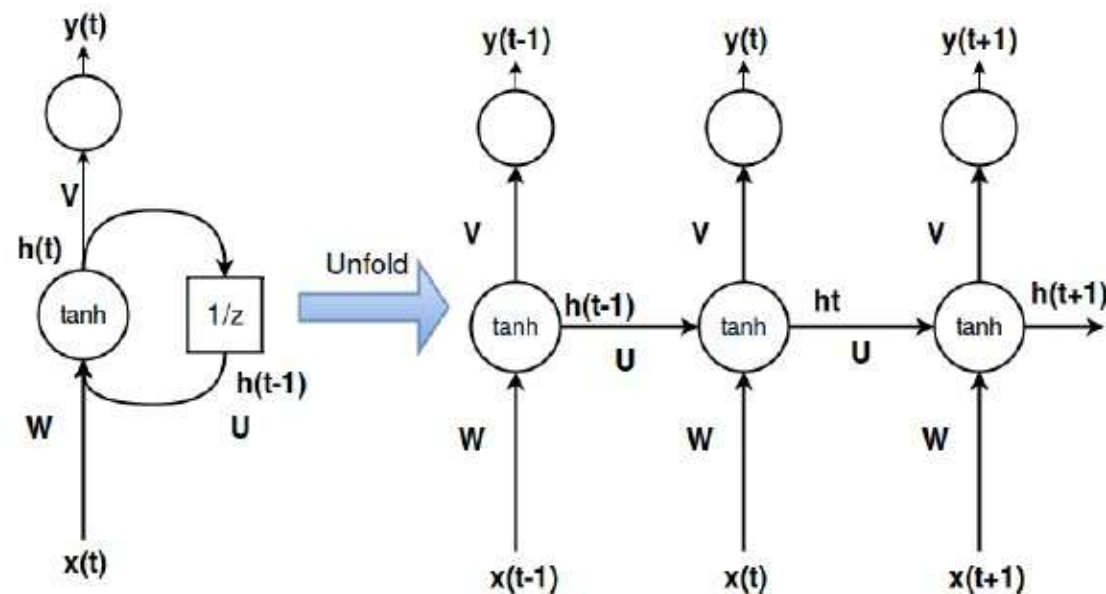
$$y_t = \text{softmax}(Vh_t)$$

$$N_{params}^i = N_{inputs}^i \times N_{units}^i + N_{units}^i \times N_{units}^i + N_{units}^i$$

$$N_{units}^i \times N_{outputs}^i$$

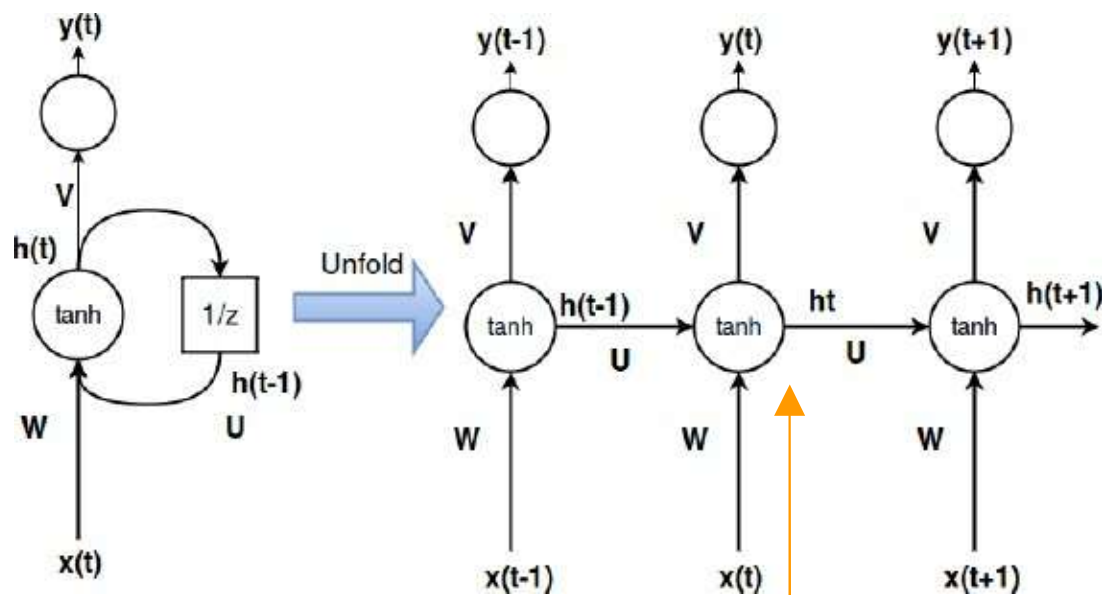
RNN: unfolding

ATTENZIONE: adesso abbiamo extra profondità! Ogni timestep è un extra livello di profondità (come uno stack più profondo di layer in una rete feed-forward!)



RNN: profondità

Propagazione in avanti **nello spazio**

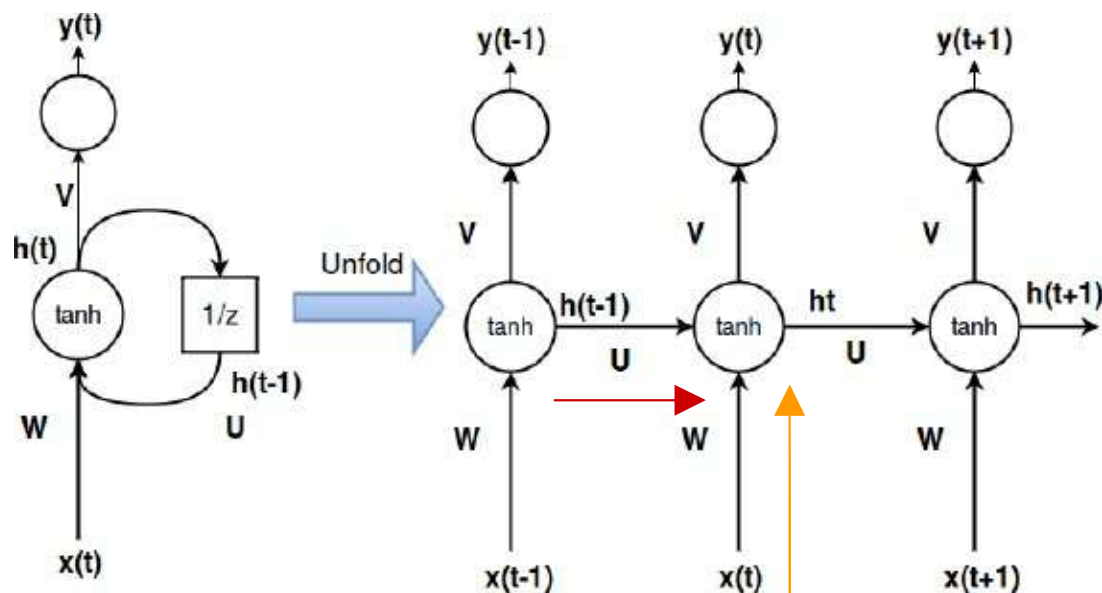


$$h_t = g(W \cdot x_t + U \cdot h_{t-1} + b_h)$$

$$x_t \in \mathbb{R}^n$$

RNN: profondità

Propagazione in avanti **nel tempo**



$$h_t = g(W \cdot x_t + U \cdot h_{t-1} + b_h)$$

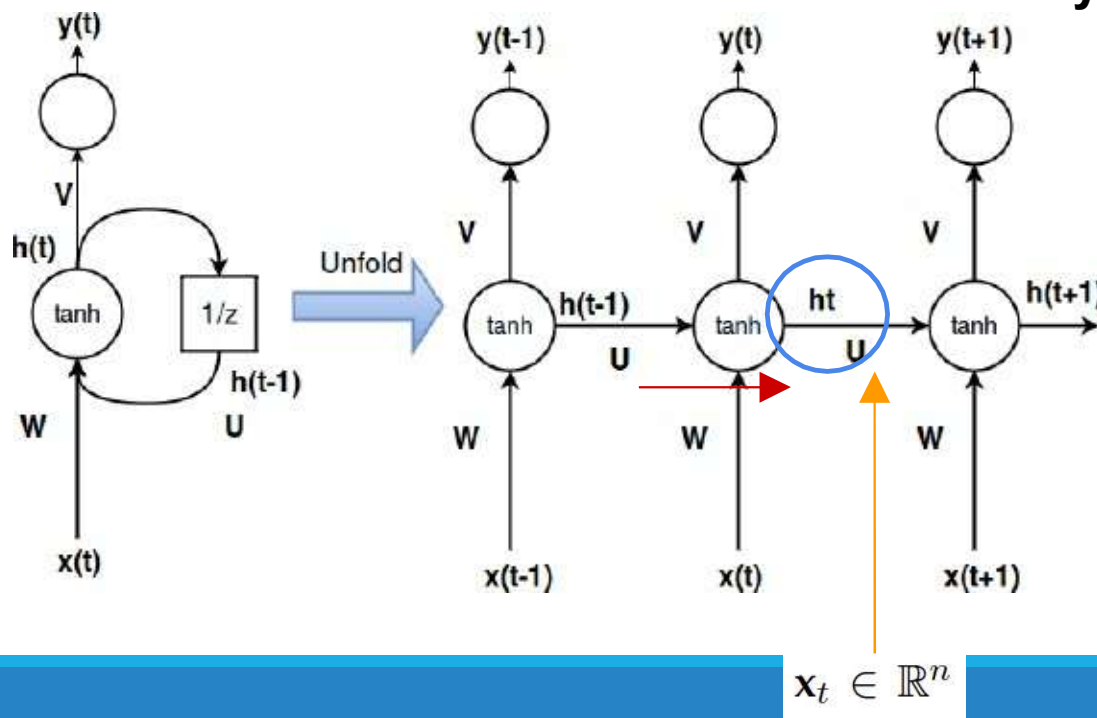
$$x_t \in \mathbb{R}^n$$

RNN: profondità In spazio + tempo

Abbiamo quindi due flussi di dati: **avanti nello spazio + tempo: 2 proiezioni per attivazione di layer**

Scorso passo temporale include il contesto delle nostre decisioni ricorsivamente

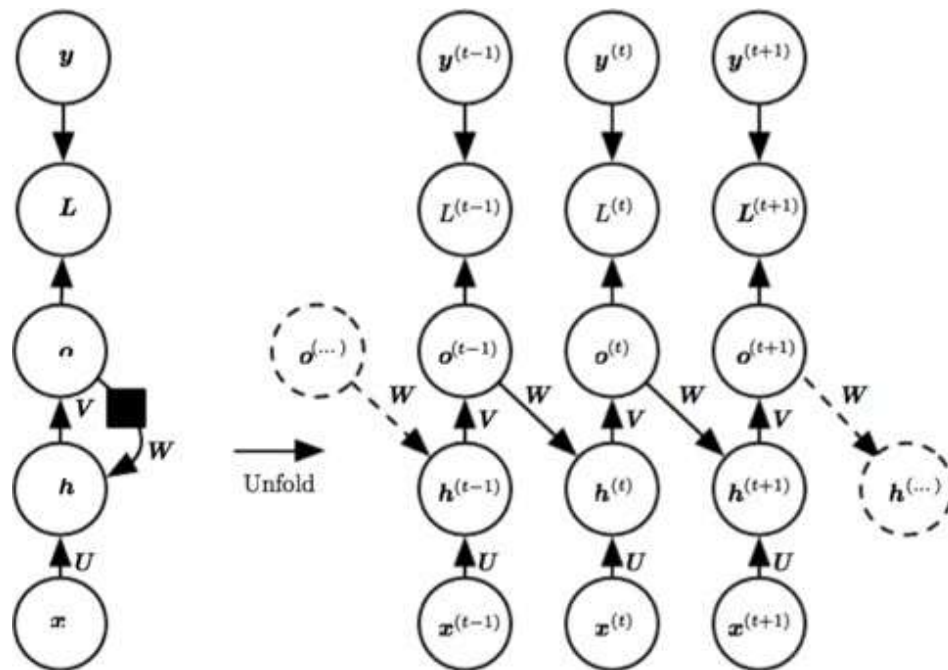
W, U, V condiviso attraverso tutti i timestep



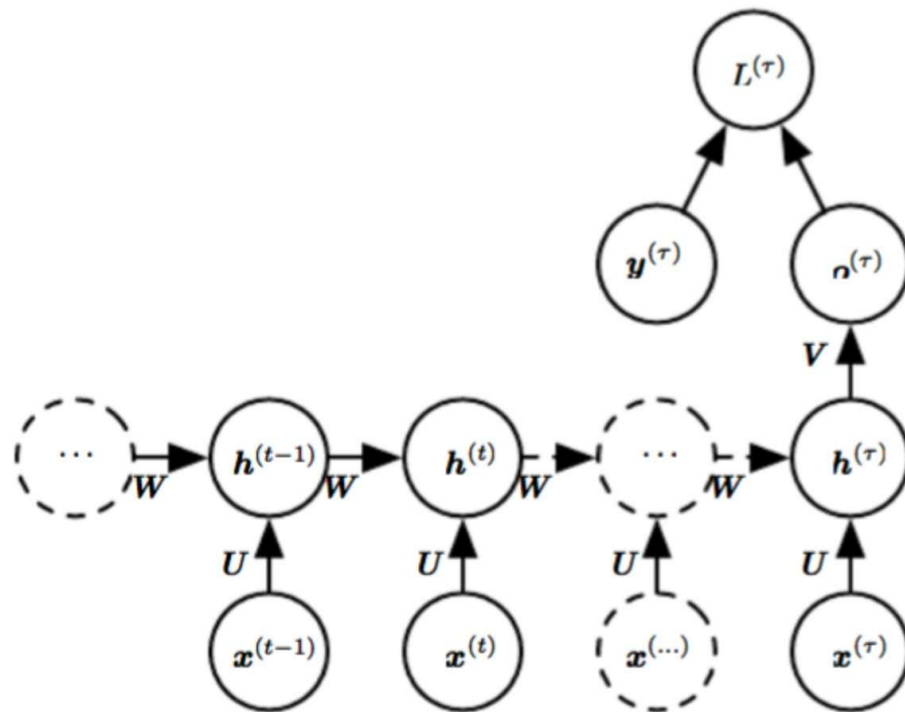
$$h_t = g(W \cdot x_t + U \cdot h_{t-1} + b_h)$$

$$y_t = \text{softmax}(Vh_t)$$

Alternative della ricorrenza



Alternative degli output: predizione di serie di output



Addestrare una RNN: BPTT

Backpropagation through time (BPTT): l'algoritmo di addestramento per l'aggiornamento dei pesi della rete per minimizzare l'errore considerando il tempo

Ricordiamo la backpropagation

1. Presentare un input di addestramento e propagarlo attraverso la rete per ottenere un output.
2. Confrontare l'output predetto con l'output atteso e calcolare l'errore.
3. Calcolare le derivate degli errori rispetto ai pesi della rete.
4. Aggiornare i pesi per minimizzare l'errore.
5. Ripetere.

BPTT

Cross Entropy Loss

$$h_t = f(\mathbf{W} \cdot \mathbf{x}_t + \mathbf{U} \cdot h_{t-1} + \mathbf{b})$$

$$y_t = \text{softmax}(Vh_t)$$

$$E_t(y_t, \hat{y}_t) = -y_t \log \hat{y}_t$$

$$\begin{aligned} E(y, \hat{y}) &= \sum_t E_t(y_t, \hat{y}_t) \\ &= - \sum_t y_t \log \hat{y}_t \end{aligned}$$

BPTT

$$\frac{\partial E}{\partial W} = \sum_t \frac{\partial E_t}{\partial W}.$$

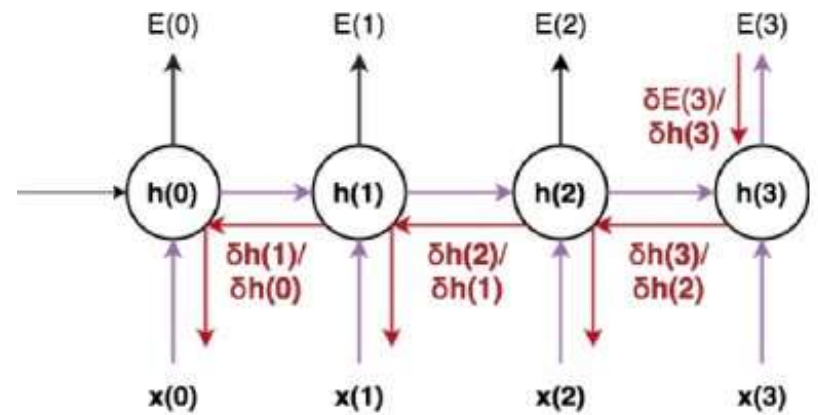
NOTA: il nostro obiettivo è calcolare i gradienti dell'errore rispetto ai parametri U, W e V e poi apprendere buoni parametri usando lo Stochastic Gradient Descent.

Proprio come sommiamo gli errori, sommiamo anche i gradienti a ogni timestep per un singolo esempio di training

BPTT

$$\frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial h_3} \frac{\partial h_3}{\partial W}$$

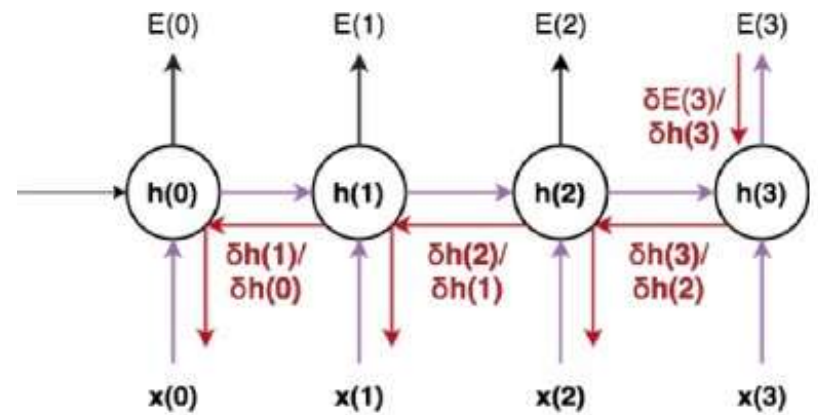
$$h_3 = f(Ux_t + Wh_2)$$



Example back-prop in time with 3 time-steps

BPTT

$$\frac{\partial E_3}{\partial W} = \sum_{h=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W}$$



Example back-prop in time with 3 time-steps

Vanishing gradient

- **Durante l'addestramento i gradienti esplodono/svaniscono facilmente a causa della profondità nel tempo**

Vanishing gradient

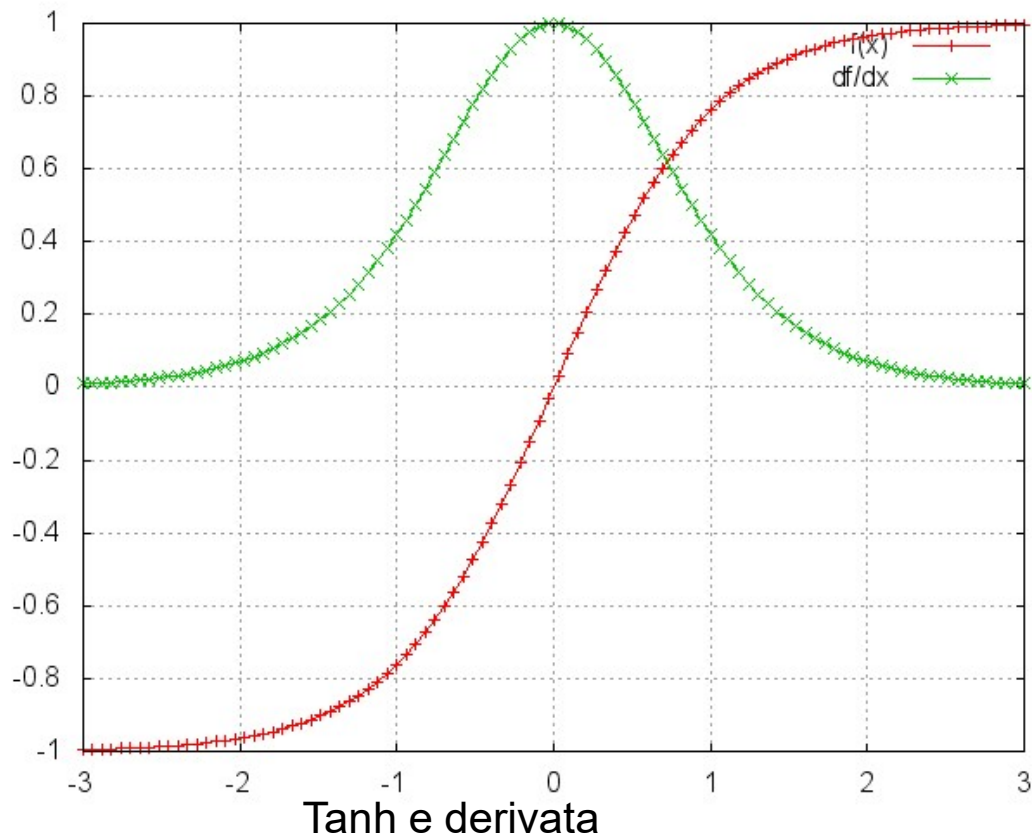
Per esempio,

$$\frac{\partial E_3}{\partial W} = \sum_{h=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W}$$

$\frac{\partial h_3}{\partial h_1} = \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1}$

$$\frac{\partial E_3}{\partial W} = \sum_{h=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial h_3} \left(\prod_{j=k+1}^3 \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W}$$

Vanishing gradient



$$\frac{\partial E_3}{\partial W} = \sum_{h=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial h_3} \left(\prod_{j=k+1}^3 \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W}$$

Soluzioni standard

Corretta inizializzazione della matrice dei pesi

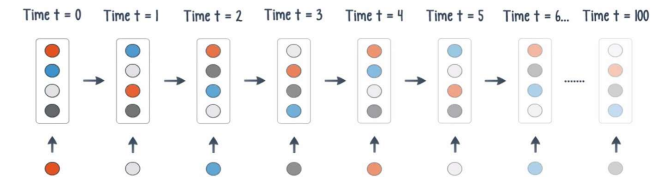
Regolarizzazione degli output O Dropout

Utilizzo di attivazioni ReLU dato che la sua derivata è sempre 0 o 1

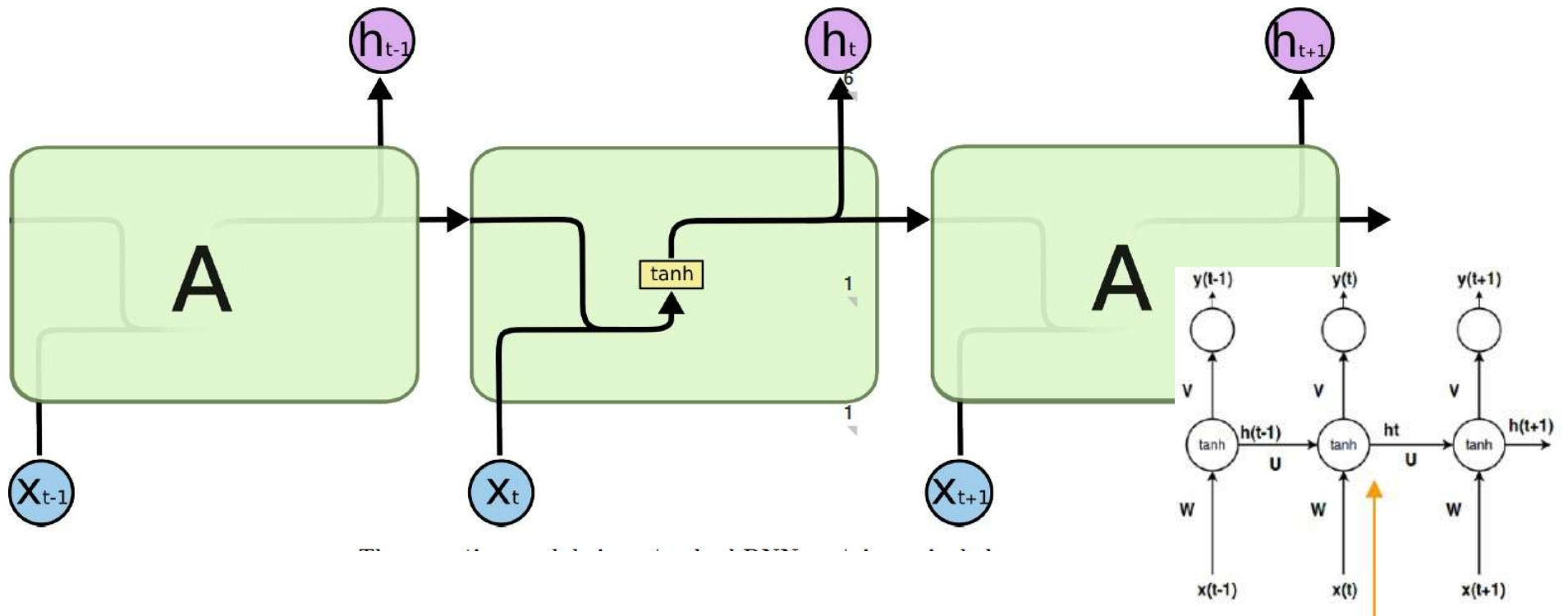
5. Metodo di gating

Standard RNN

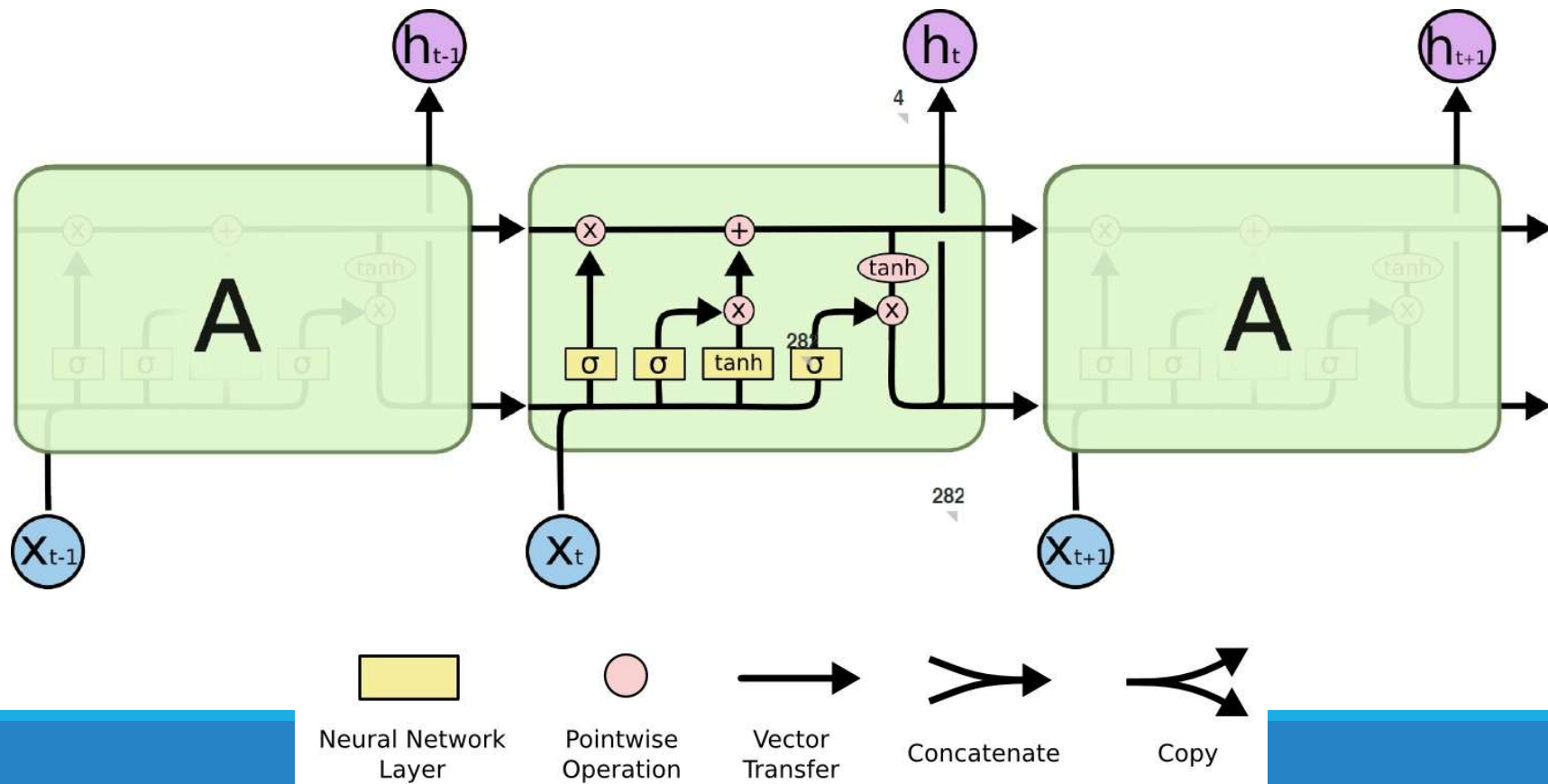
Decay of information through time



<https://www.nextbigfuture.com/2016/03/recurrent-neural-nets.html>



Long-Short Term Memory (LSTM)



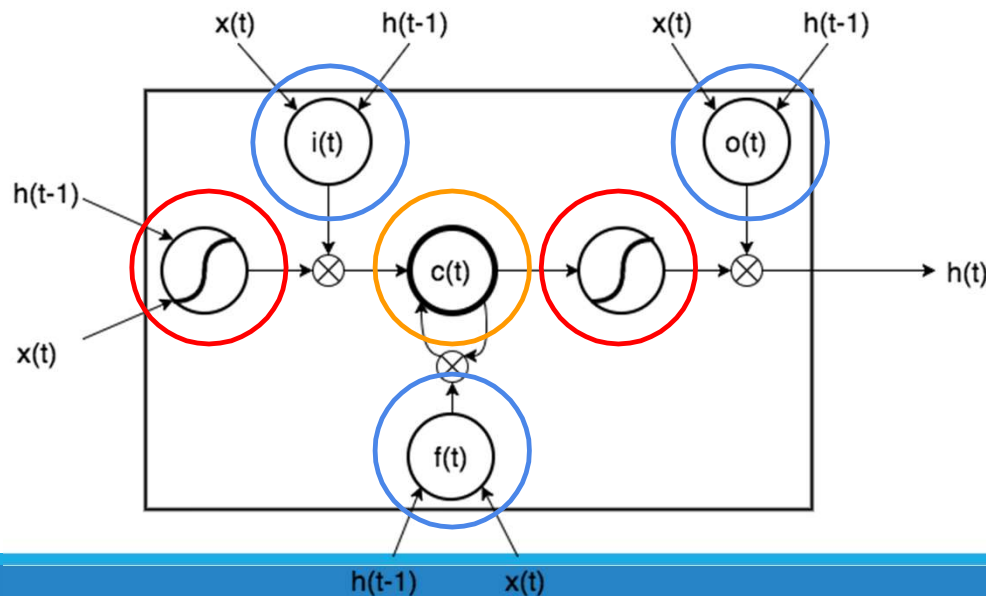
Metodo di gating

1. Modifica il modo in cui l'informazione passata è preservata → crea la nozione di **stato di memoria (cell state)**, che mantiene informazioni a lungo termine in un modo più sicuro
2. **Rendi ogni unità RNN capace di decidere se le informazioni del timestep attuale importano o meno**, per accettarle o scartarle
3. **Rendi ogni unità RNN in grado di dimenticare ciò non è più utile** rimuovendo quell'informazione dal cell state
4. **Rendi ogni unità RNN in grado di restituire le decisioni nel momento in cui è pronta a farlo**

Cella LSTM (Long Short-Term Memory)

Una cella LSTM è definita da due gruppi di neuroni più il cell state (unità di memoria):

- **Gates**
- **Unità di attivazione**
- **Cell state**



$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

$$\hat{C}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

$$C_t = i_t \odot \hat{C}_t + f_t \odot C_{t-1}$$

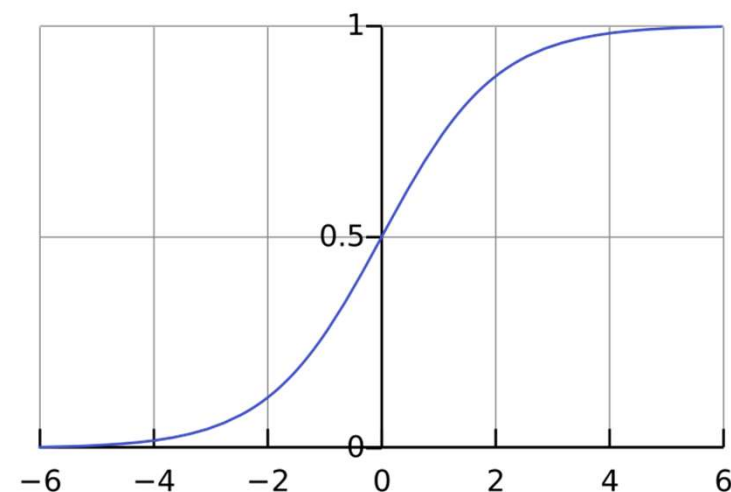
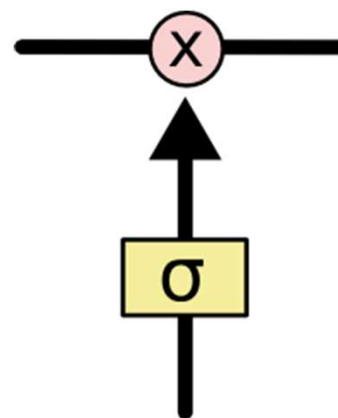
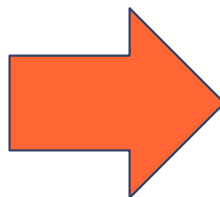
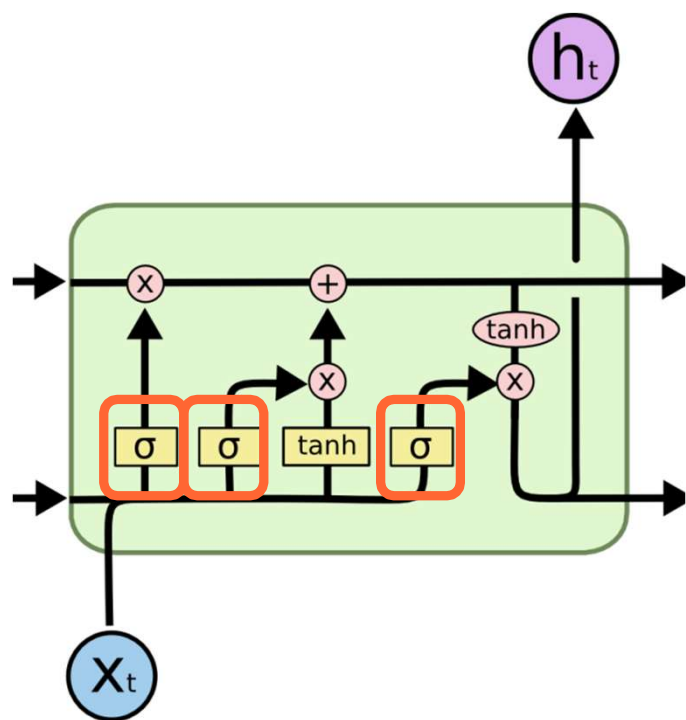
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

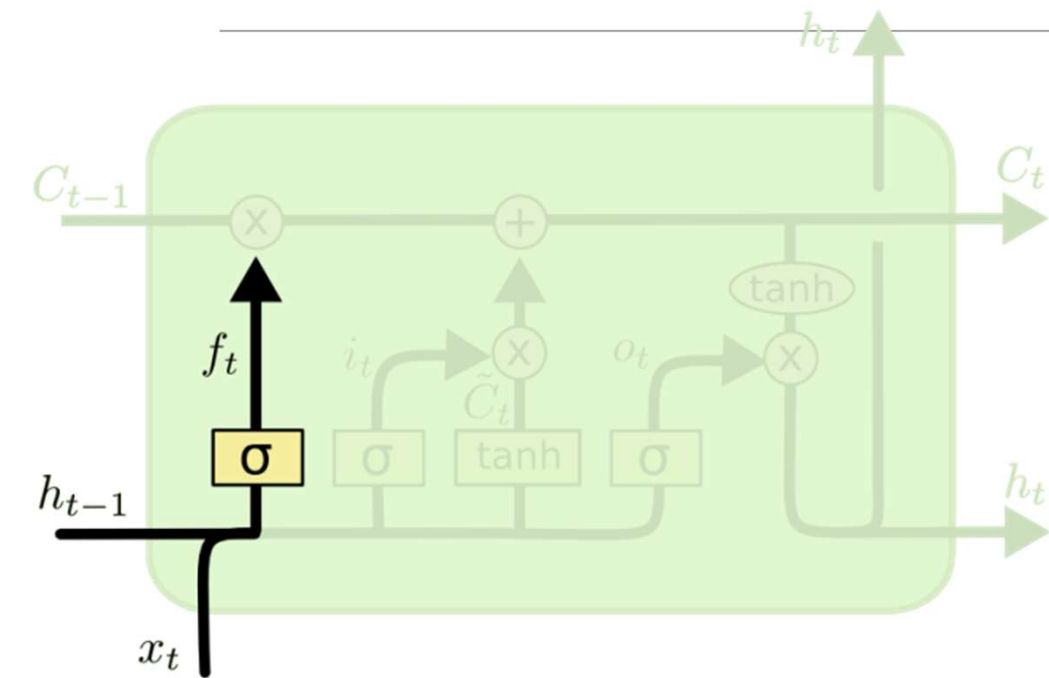
Flusso di
calcolo

Cella LSTM (Long Short-Term Memory)

Tre **gate** sono governati da unità *sigmoide* (quindi $[0,1]$) che controllano l'ingresso e uscita di informazioni



Forget Gate

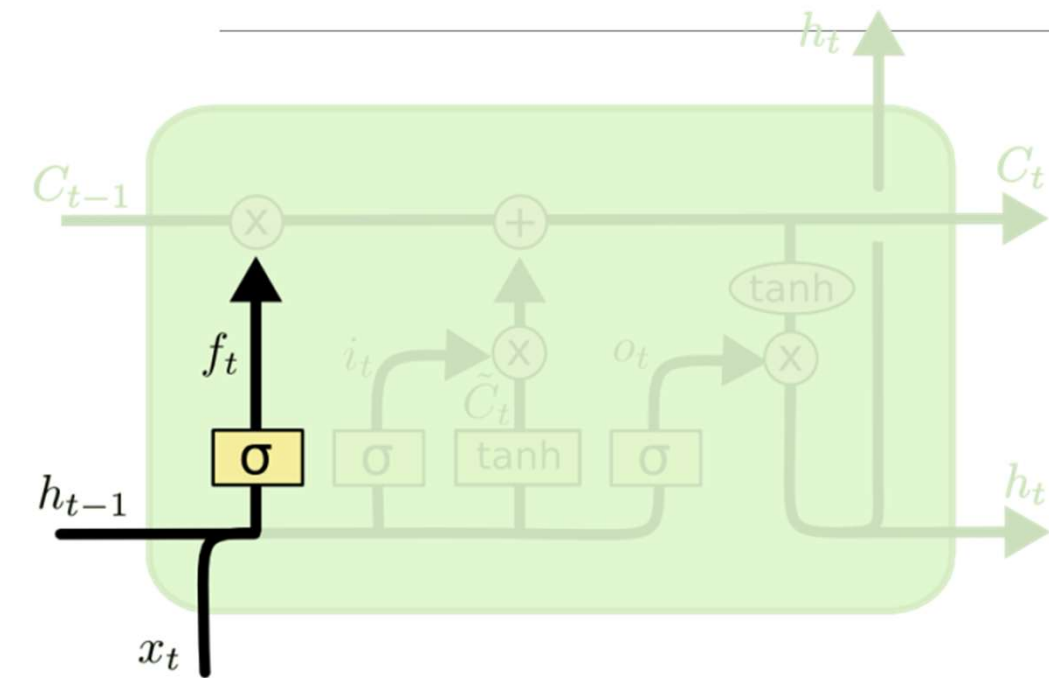


Forget Gate

$$f_t = \sigma (W_f \cdot \underbrace{[h_{t-1}, x_t]}_{\text{Concatenazione}} + b_f)$$

Concatenazione

Forget Gate: Esempio



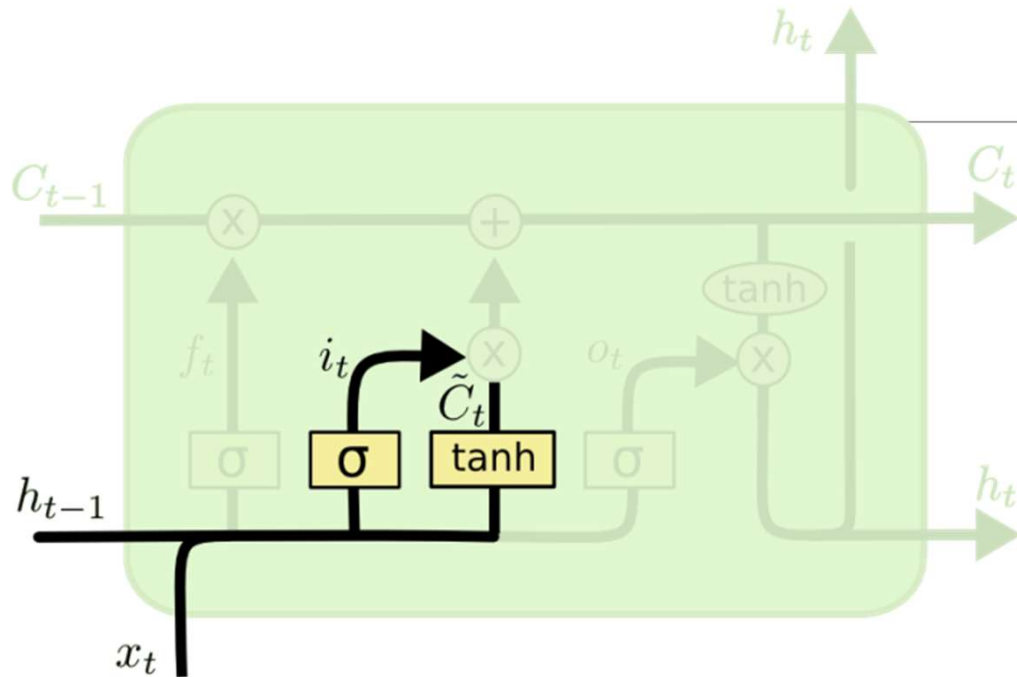
$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

MODELLAZIONE DEL LINGUAGGIO

Giovanni è un ragazzo attivo e Anna è una ragazza calma

Dimenticati del genere "maschile"

Input Gate



Input Gate

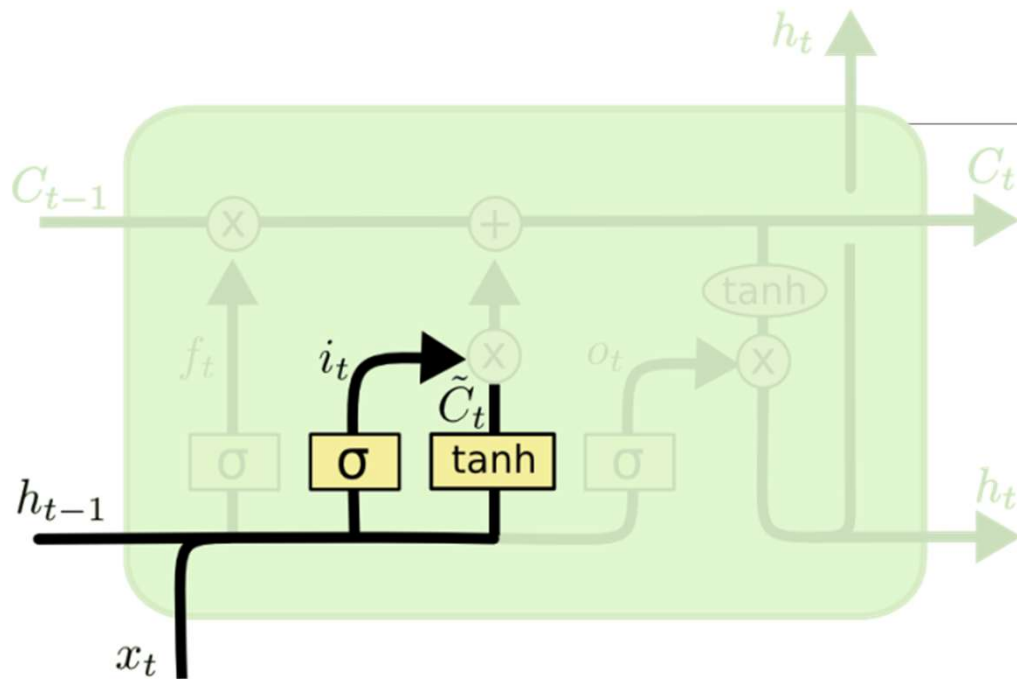
$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$

Nuovo contributo al cell state

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Neurone classico

Input Gate: Esempio



Input Gate

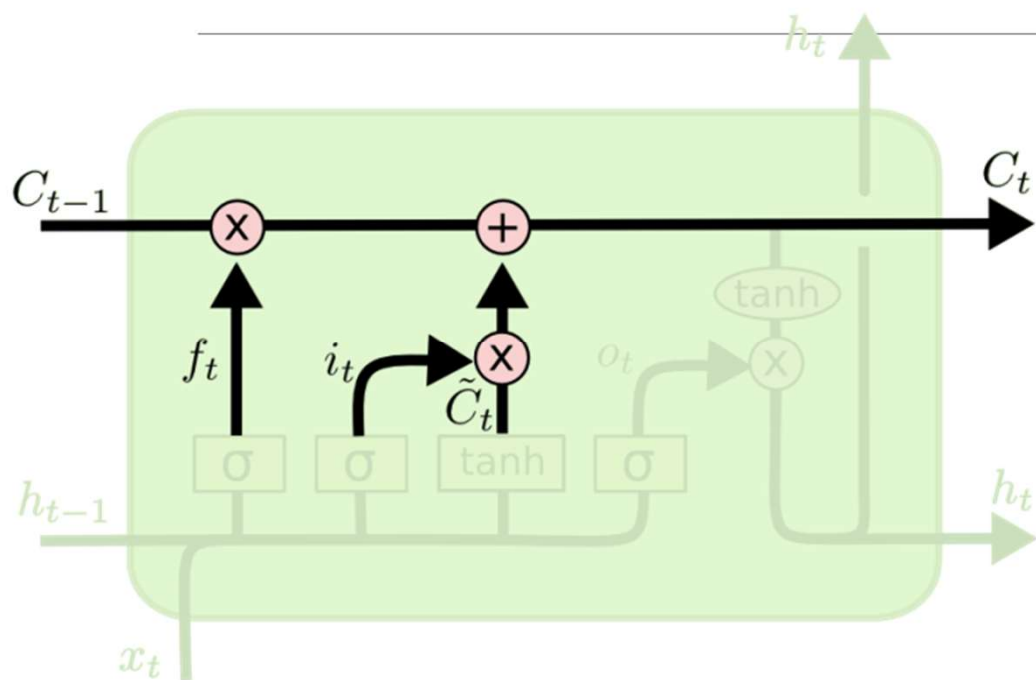
$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$

MODELLAZIONE DEL LINGUAGGIO

Giovanni è un ragazzo attivo e **Anna** è una ragazza calma

Fai entrare l'informazione riguardo genere "femminile"

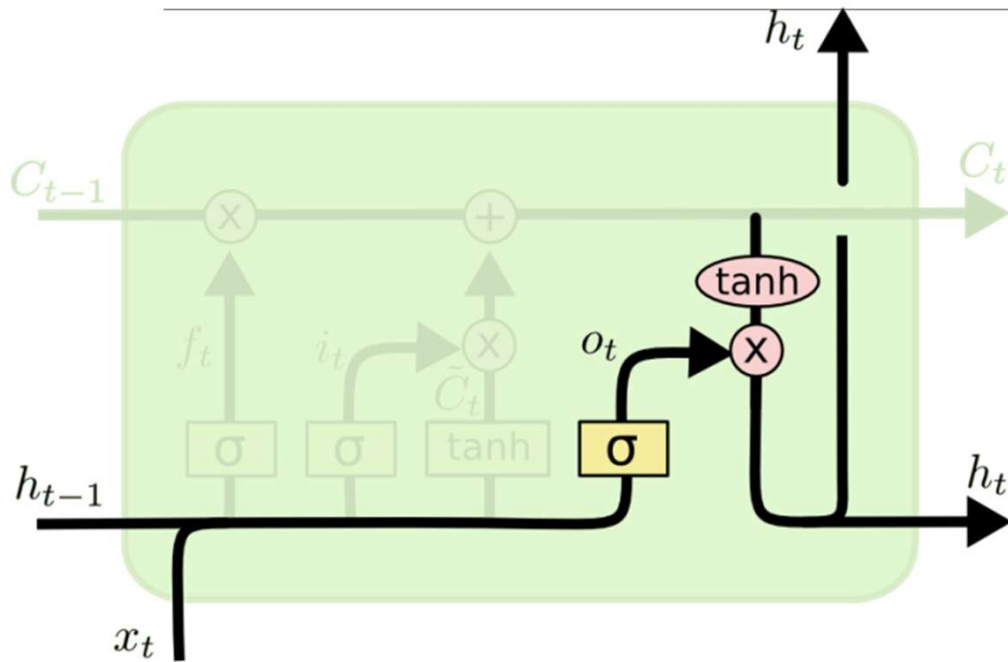
Aggiorna il Cell State



Aggiorna il Cell State (memoria):

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output Gate



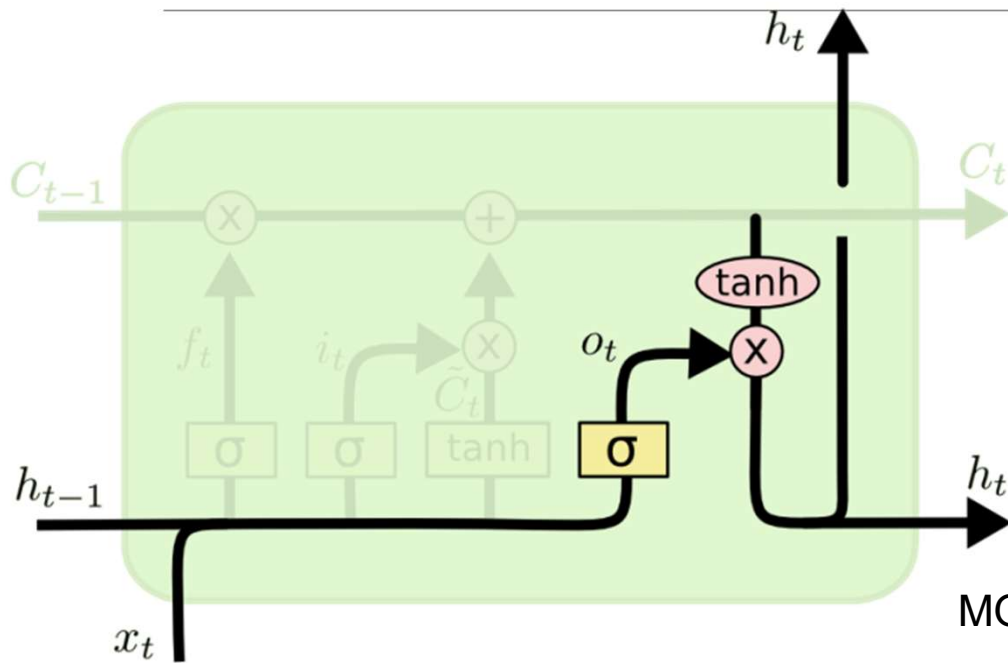
Output Gate

$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

Output al prossimo layer

$$h_t = o_t * \tanh (C_t)$$

Output Gate: Esempio



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

MODELLAZIONE DELLA LINGUA

Giovanni è un ragazzo attivo e Anna è una ragazza calma

Info rilevanti per un verbo? "femminile", "singolare", "3° persona"

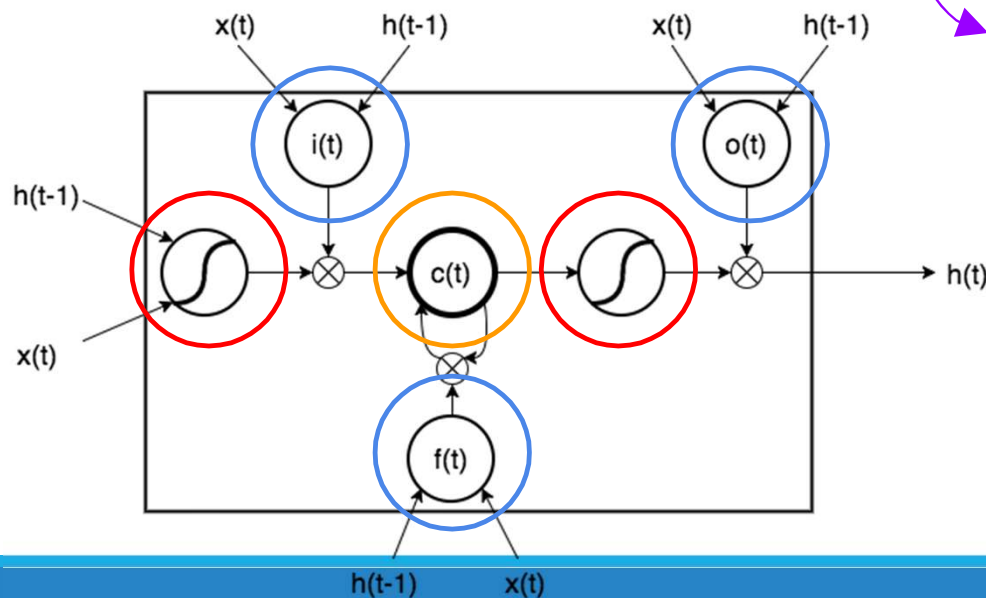
LSTM: parametri

Una cella LSTM è definita da due gruppi di neuroni più il cell state (unità di memoria):

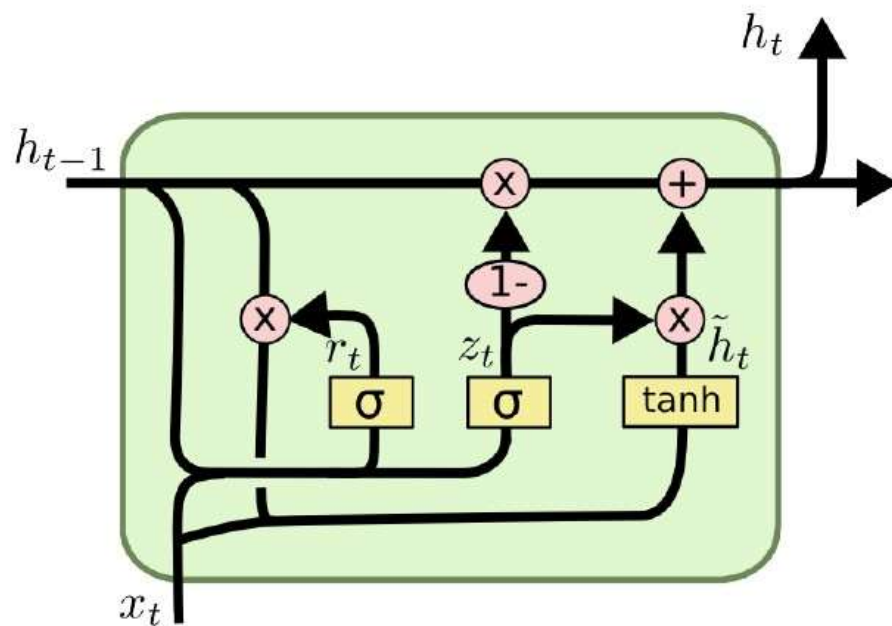
- **Gates**
- **Unità di attivazione**
- **Cell state**

$$N_{params}^i = 4 \times (N_{inputs}^i \times N_{units}^i + N_{units}^i \times N_{units}^i + N_{units}^i)$$

3 gate + attivazione dell'input



Gated Recurrent Unit (GRU)



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

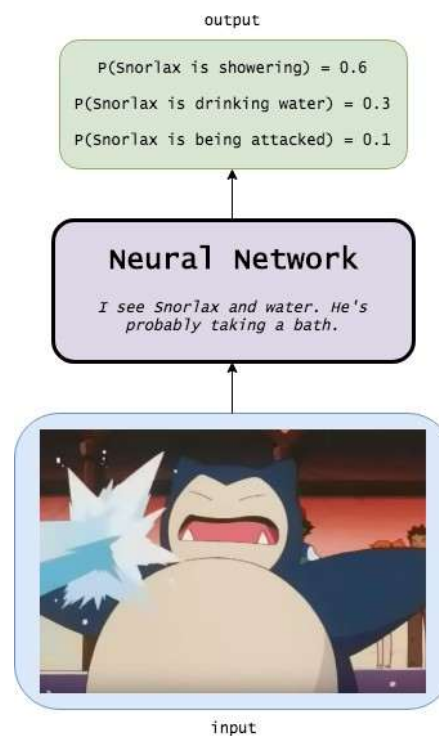
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

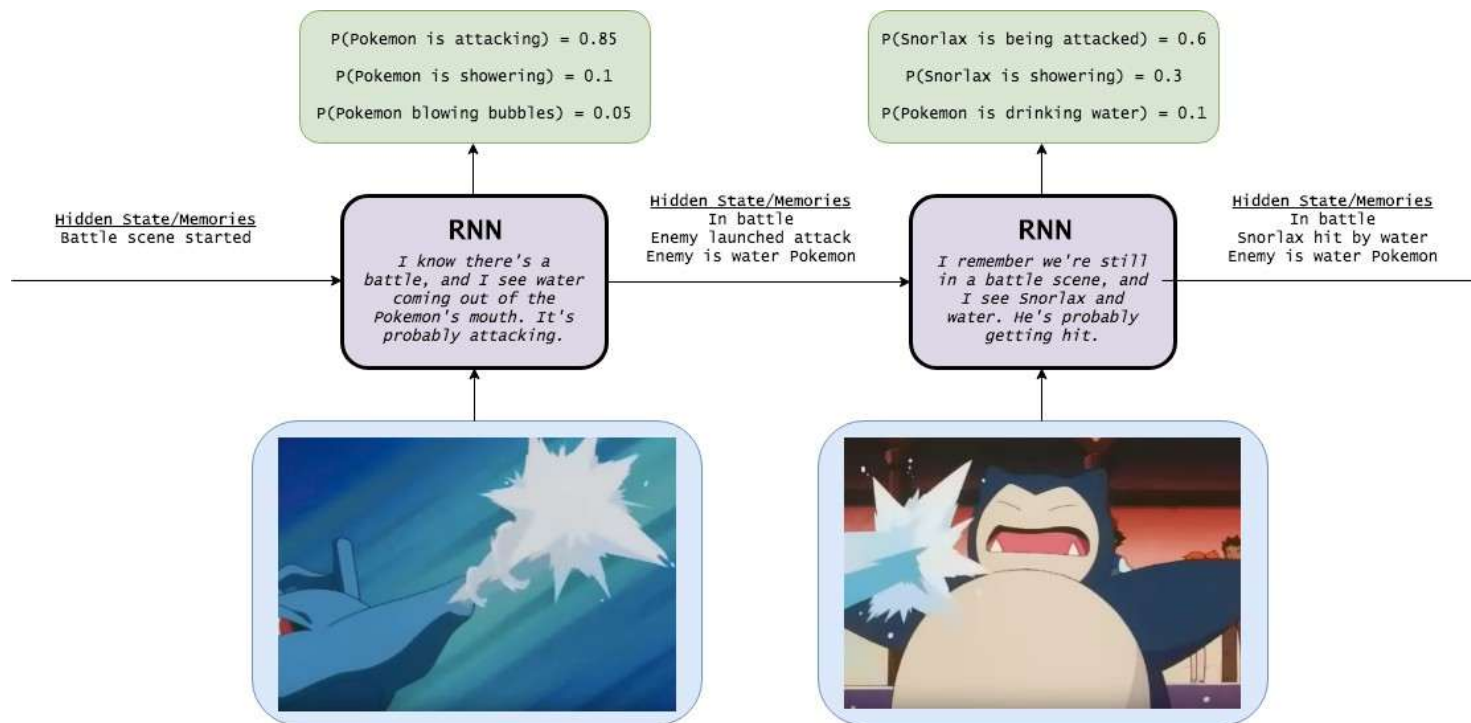
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

$$N_{params}^u = 3 \times (N_{inputs}^u \times N_{units}^u + N_{units}^u \times N_{units}^u + N_{units}^u)$$

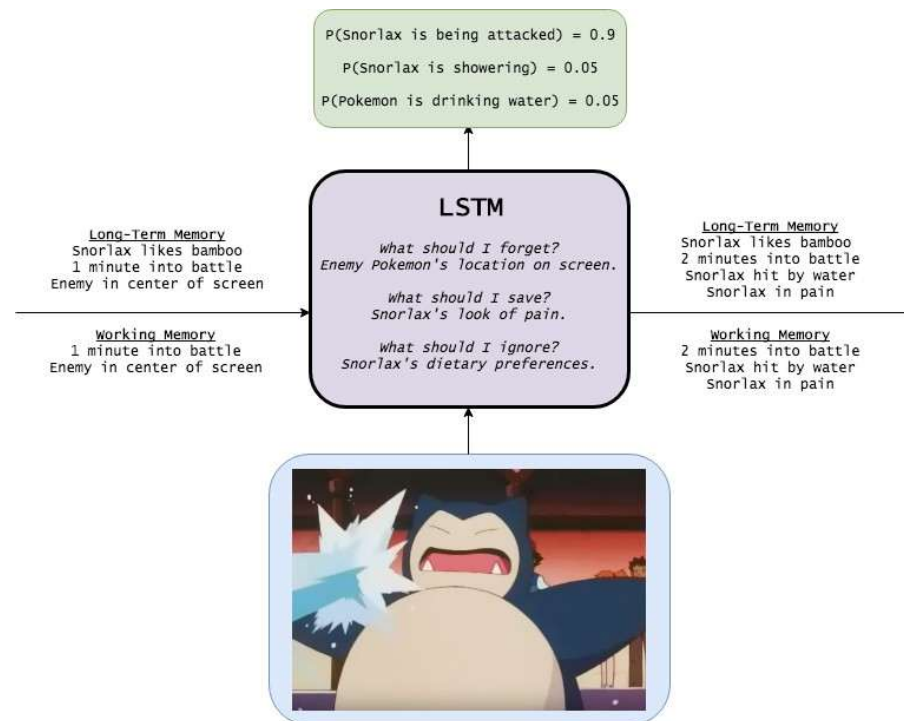
Confronto visivo tra **FNN**, RNN base e LSTM



Confronto visivo tra **FNN**, RNN base e LSTM

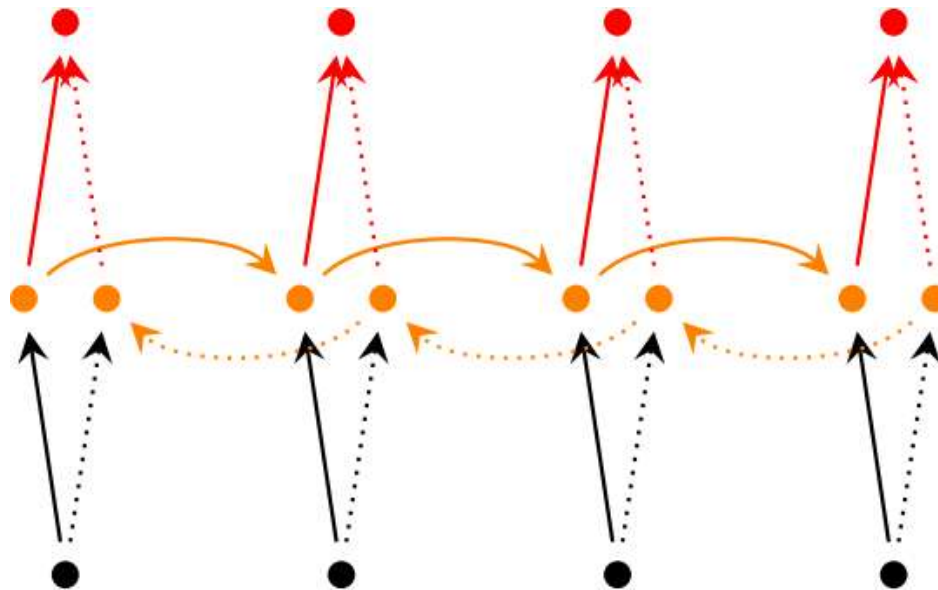


Confronto visivo tra FNN, RNN base e LSTM

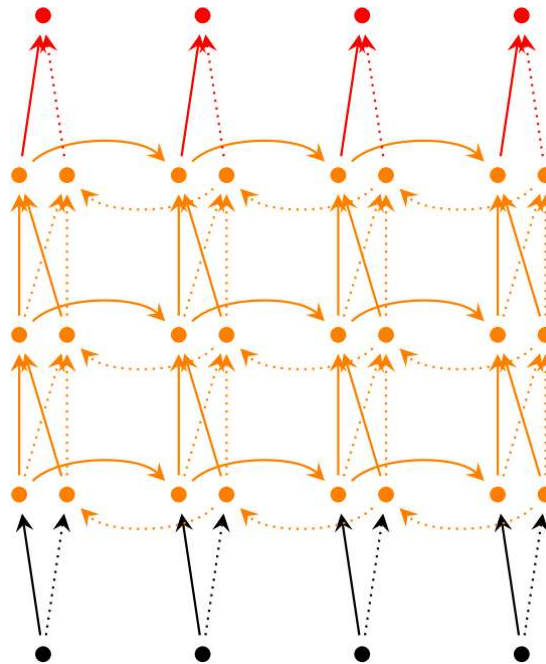


6. Altre estensioni dell'RNN

RNN Bidirezionale



Deep RNN



7. Casi d'uso

Language models a livello di carattere

$$P('he')=P(e|h)$$

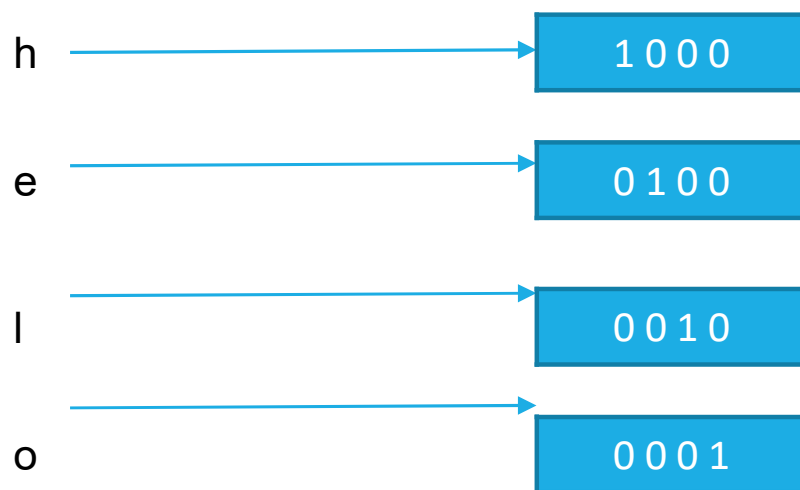
$$P('hel')=P(l|'he')$$

$$P('hell')=P(l|'hel')$$

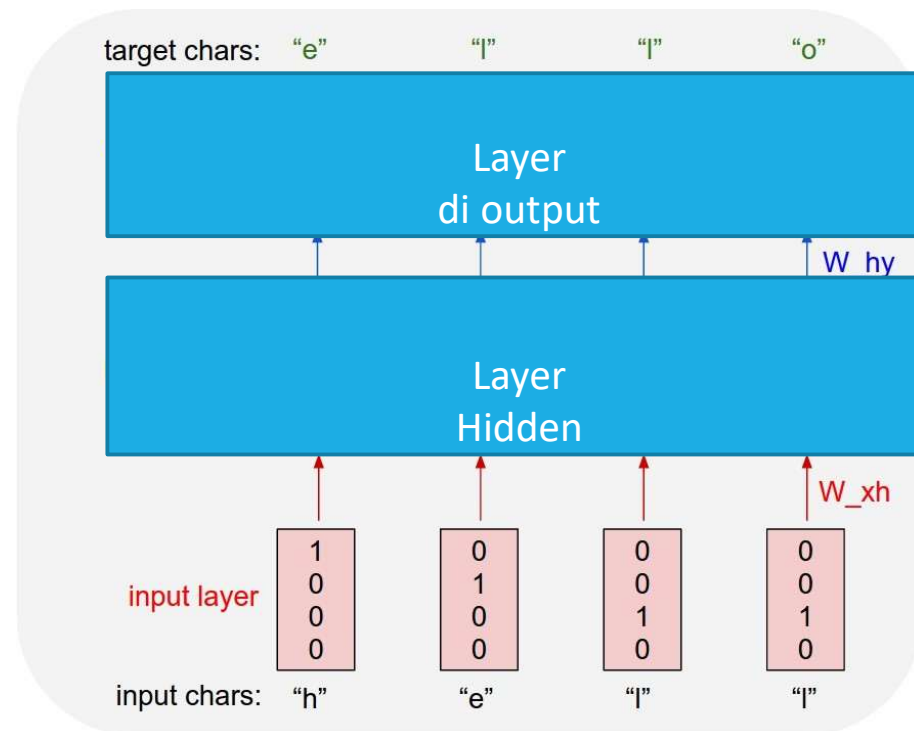
$$P('hello')=P(o|'hell')$$

→ Tutte queste probabilità dovrebbero essere plausibili

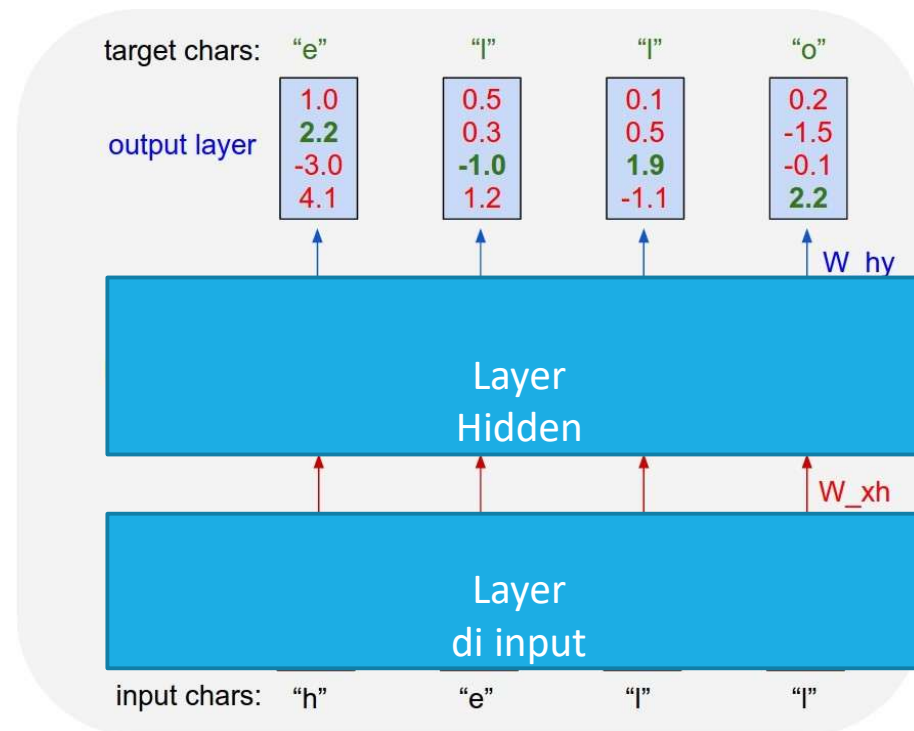
Language models a livello di carattere con RNN



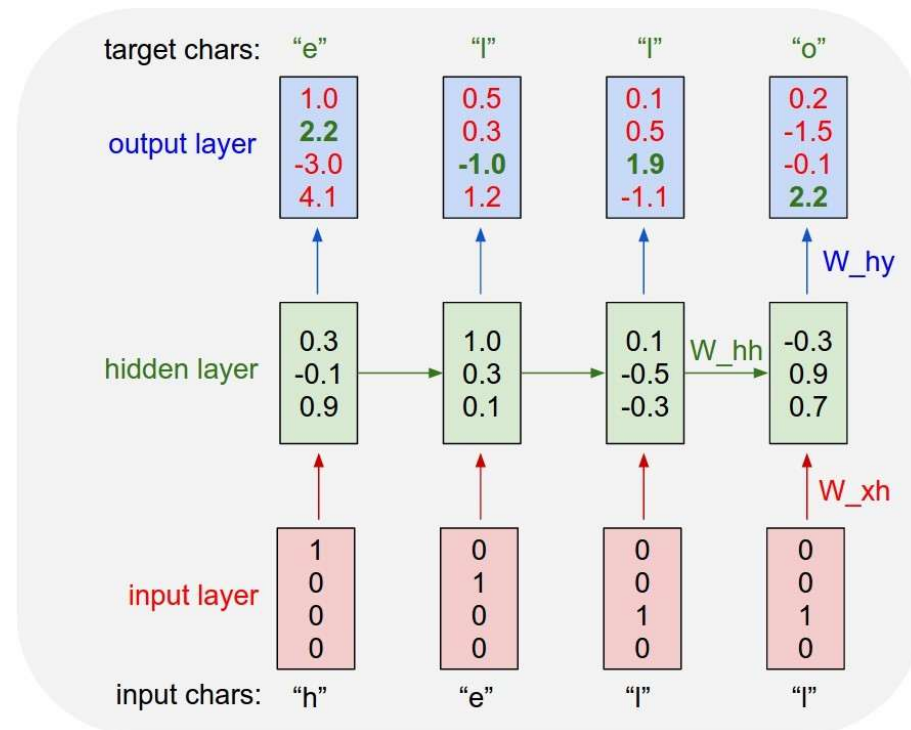
Language models a livello di carattere



Language models a livello di carattere



Language models a livello di carattere



Grazie!
Domande?

A solid blue horizontal bar spanning the width of the slide, located at the bottom.